

LOCAL AI CODE EDITOR

Product Requirements Document (PRD)

Version 1.0 | May 2025

Project Name	Local AI Code Editor
Author	Sumeet
Version	1.0
Status	Draft
Target Platform	Linux / Windows / macOS (via Tauri)

1. Executive Summary

Local AI Code Editor is a fully offline, open-source code editor powered by local AI models. Built for developers who cannot or do not want to pay for AI coding subscriptions, it brings Copilot/Cursor-level AI assistance entirely to the user's own machine — no internet required, no data leaving the device.

The editor integrates directly with Ollama-managed models and raw .gguf files via llama.cpp, enabling AI-powered chat, inline code completion, and autonomous file/terminal operations — all without any cloud dependency.

2. Problem Statement

Modern AI coding tools like GitHub Copilot, Cursor, and Windsurf require paid subscriptions ranging from \$10–\$20/month. This creates a barrier for:

- Students and developers in regions with limited purchasing power
- Privacy-conscious developers who don't want their code sent to cloud servers
- Offline or restricted-network environments
- Developers who already have capable local hardware (GPU-equipped machines)

Existing local LLM tools (like Ollama) are powerful but lack a proper IDE integration. This project bridges that gap.

3. Goals & Objectives

3.1 Primary Goals

- Build a fully functional code editor with VS Code-style UI
- Integrate local AI models (.gguf via llama.cpp and Ollama) for coding assistance
- Enable AI to make real file edits and run terminal commands
- Provide a settings page for managing AI models and permissions
- Ensure zero cloud dependency — everything runs locally

3.2 Non-Goals (v1.0)

- Cloud model integration (OpenAI, Anthropic, etc.) — may be added later
- Mobile support
- Collaborative real-time editing (multiplayer)
- Plugin/extension marketplace

4. Target Users

User Type	Description	Pain Point Solved
Student Developer	CS students with limited budget	No subscription needed
Privacy-Focused Dev	Devs who don't want code sent to cloud	All data stays local
GPU Owner	Has RTX/AMD GPU, wants to use it for AI	Full hardware utilization
Offline Developer	Works in restricted network environments	No internet required

5. Features & Requirements

5.1 Core Editor (P0 — Must Have)

#	Feature	Details
1	Monaco Editor Integration	VS Code's editor engine embedded in app
2	File Open / Save	Open single files and folders; save with Ctrl+S
3	File Explorer Sidebar	Left panel showing folder/file tree
4	Syntax Highlighting	All major languages via Monaco's built-in support
5	Tabs	Multiple open files as tabs
6	Integrated Terminal	Built-in terminal panel at bottom

5.2 AI Chat Sidebar (P0 — Must Have)

#	Feature	Details
1	AI Chat Panel	Right-side panel for conversing with AI model
2	Ollama Model Support	Connect to locally running Ollama instance
3	.gguf Model Support	Load raw .gguf files via llama.cpp backend
4	Model Switcher Dropdown	Switch between available models mid-conversation
5	Current File Context	Active file content auto-included as context
6	@ File Mention	Type @filename to include any file in context
7	Code Block Rendering	AI responses render code with syntax highlighting

5.3 AI-Powered File & Terminal Actions (P1 — High Priority)

#	Feature	Details
1	AI File Edits	AI can propose and apply changes directly to open files
2	Diff View	Show accept/reject diff before applying AI changes
3	Terminal Commands	AI can suggest shell commands; user approves before execution
4	Auto-apply Toggle	Optional setting to auto-accept AI file changes

5.4 Settings Page (P1 — High Priority)

#	Feature	Details
1	Ollama Endpoint Config	Set custom Ollama host URL (default: localhost:11434)
2	.gguf File Loader	Browse and load local .gguf model files
3	Model Permissions	Toggle: allow file edits, allow terminal commands

4	Context Window Size	Configure max tokens sent to model
5	Theme Selection	Light/dark mode
6	Editor Preferences	Font size, tab size, word wrap, etc.

5.5 Inline Code Completion (P2 — Nice to Have)

#	Feature	Details
1	Ghost Text Completion	Inline suggestions appear as user types (Tab to accept)
2	FIM Model Support	Uses Fill-in-Middle models like Qwen2.5-Coder
3	Completion Toggle	Enable/disable inline completion in settings

6. Recommended Tech Stack

Layer	Technology	Reason
App Framework	Tauri (Rust + React)	Lightweight vs Electron; aligns with Rust learning
Editor Engine	Monaco Editor	VS Code's engine; free & embeddable
UI	React + TypeScript	Component-based, type-safe, fast iteration
AI Backend (Local)	Ollama REST API	Easy local model management; already familiar
AI Backend (.gguf)	llama.cpp (Rust binding)	Direct .gguf file inference without Ollama
Terminal	Xterm.js	Full-featured browser-based terminal
Styling	Tailwind CSS	Utility-first, fast to prototype

7. Recommended AI Models

Use Case	Model	Size	VRAM (approx.)
AI Chat / Debug	llama3.2:3b (Ollama)	~2 GB	~2.5 GB
Code Completion (FIM)	qwen2.5-coder:1.5b	~1 GB	~1.5 GB
Powerful Chat	gemma3:4b (Ollama)	~3 GB	~3.5 GB
Custom .gguf	Any GGUF model	Varies	Depends on quant

Note: Tested against RTX 3050 (4GB VRAM). Models up to 3B parameters run smoothly in Q4 quantization.

8. Build Phases

Phase	Name	Deliverables	Duration (est.)
Phase 1	Core Editor	Tauri setup, Monaco editor, file open/save, file explorer, tabs	2 weeks
Phase 2	AI Chat Sidebar	Ollama integration, chat UI, file context, model switcher	2 weeks
Phase 3	AI Actions	File edits with diff view, terminal command suggestions	2 weeks
Phase 4	Settings Page	Model config, permissions, editor preferences	1 week
Phase 5	.gguf Support	llama.cpp backend, .gguf file loader	1–2 weeks
Phase 6	Inline Completion	Ghost text, FIM model integration	1–2 weeks

9. UI Layout

The editor follows a 3-panel layout inspired by VS Code:

LEFT PANEL File Explorer Project tree, open files	CENTER Monaco Editor Code tabs, syntax highlighting, diff view	RIGHT PANEL AI Chat Sidebar Chat, model switcher, context display
BOTTOM — Integrated Terminal (xterm.js)		

10. Success Metrics

Metric	Target
AI response latency (3B model, RTX 3050)	< 3 seconds first token
App startup time	< 2 seconds
.gguf model load time	< 10 seconds
File edit diff display	< 500ms after AI responds
Inline completion trigger delay	< 800ms after typing stops

11. Open Questions

- Should .gguf inference use a separate Rust sidecar process or a pre-built llama.cpp server binary?
 - What is the minimum supported VRAM? (Currently targeting 4GB — RTX 3050)
 - Should the app support HuggingFace Inference API as an optional cloud fallback when no local model is available?
 - How should the app handle model download directly (pull from HuggingFace Hub) vs user-provided files?
 - What license should this project ship under? (MIT recommended for open-source visibility)
-